

UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN Y TECNOLOGÍA DE LA INFORMACIÓN

INGENIERIA CIVIL EN INFORMATICA

Tarea 1

Frontera de Pareto o Skyline

Integrantes:

Maximiliano Alejandro Riquelme Baeza
Matías Antonio Constanzo Monsalve
Juan Pedro Recabal Rodríguez

Profesor: Gilberto Gutiérrez

Índice

1. Introducción **2**

2. Algoritmos Implementados **2**
2.1. Alg1: Algoritmo Ingenuo 2
2.2. Alg2Arreglo: Algoritmo Incremental con Arreglo 3
2.3. Alg3MiEstructura: Optimización mediante Ordenamiento 3

3. Análisis de Complejidad **4**

4. Implementación **5**
4.1. skyline.py – Implementación de los algoritmos 5
4.2. test_skyline.py – Pruebas unitarias 7
4.3. experiments.py – Script de experimentos 7

5. Resultados Experimentales **8**

6. Conclusiones **9**

1. Introducción

Este informe presenta el desarrollo de algoritmos para encontrar la Frontera de Pareto (*Skyline*) de un conjunto de puntos $S \subset \mathbb{R}^2$. El objetivo es identificar los puntos que no son dominados por ningún otro en el conjunto, asumiendo que "mejor" significa minimizar ambas coordenadas.

Formalmente, dado un conjunto de puntos $S \subset \mathbb{R}^2$, se dice que un punto p domina a un punto q (denotado $p \prec q$) si:

$$p_i \leq q_i \quad \text{para todo } i = 1, \ldots, n \quad \text{y} \quad p_i < q_i \quad \text{para al menos un } i.$$

Es decir, p es igual o mejor que q en todas las dimensiones, y estrictamente mejor en al menos una. La Frontera de Pareto de S se define como:

$$\text{Pareto}(S) = \{\, p \in S \mid \nexists\, q \in S \text{ tal que } q \prec p \,\}.$$

En este trabajo se implementan y comparan tres algoritmos distintos para calcular Pareto(S) en el caso bidimensional: un algoritmo ingenuo de fuerza bruta (**Alg1**), un algoritmo incremental basado en un conjunto de candidatos almacenado en un arreglo (**Alg2Arreglo**), y una optimización basada en ordenamiento (**Alg3MiEstructura**). Se presenta el análisis teórico de complejidad de cada uno, su implementación, y los resultados experimentales obtenidos al variar el tamaño del conjunto de entrada.

2. Algoritmos Implementados

A continuación se describe cada algoritmo junto con su pseudocódigo y análisis conceptual. La operación relevante utilizada para medir el costo de cada algoritmo es la comparación de dominancia $p \prec q$.

2.1. Alg1: Algoritmo Ingenuo

Este algoritmo compara cada punto p contra todos los demás puntos q del conjunto. Si existe algún q tal que $q \prec p$, entonces p es descartado.

Algorithm 1 Alg1 – Algoritmo Ingenuo

```
1:  function ALG1(S)
2:      Pareto ← ∅
3:      for cada p ∈ S do
4:          dominado ← Falso
5:          for cada q ∈ S, q ≠ p do
6:              if q ≺ p then
7:                  dominado ← Verdadero
8:                  break
9:              end if
10:         end for
11:         if no dominado then
12:             Pareto ← Pareto ∪ {p}
13:         end if
14:     end for
15:     return Pareto
16: end function
```

Complejidad: $\theta(n^2)$ siempre, ya que realiza dos bucles anidados sobre los n puntos, independiente de la configuración de los datos.

2.2. Alg2Arreglo: Algoritmo Incremental con Arreglo

Mantiene una lista C de candidatos, inicialmente vacía. Para cada punto $p \in S$ se compara contra los puntos $q \in C$:

1. Si algún $q \in C$ domina a p, se descarta p.
2. En caso contrario, se eliminan de C todos los puntos q que sean dominados por p, y luego se agrega p a C.

Algorithm 2 Alg2Arreglo – Algoritmo Incremental con Arreglo

```
1:  function ALG2ARREGLO(S)
2:      C ← ∅                                          ▷ arreglo de candidatos
3:      for cada p ∈ S do
4:          ARemover ← ∅
5:          dominado ← Falso
6:          for cada q ∈ C do
7:              if q ≺ p then
8:                  dominado ← Verdadero
9:                  break
10:             else if p ≺ q then
11:                 ARemover ← ARemover ∪ {q}
12:             end if
13:         end for
14:         if no dominado then
15:             C ← (C \ ARemover) ∪ {p}
16:         end if
17:     end for
18:     return C
19: end function
```

Análisis: en el peor caso (cuando todos los puntos pertenecen a la frontera) sigue siendo $O(n^2)$, pues C crece hasta tamaño n. Sin embargo, en la práctica $|C|$ suele ser mucho menor que n, lo que mejora significativamente el rendimiento respecto a Alg1.

2.3. Alg3MiEstructura: Optimización mediante Ordenamiento

Para el caso 2D se propone ordenar los puntos por la coordenada x (y, en caso de empate, por y). Una vez ordenados, se recorre el conjunto manteniendo el valor mínimo de y encontrado hasta el momento (min_y). Un punto pertenece a la frontera si su coordenada y es estrictamente menor que el min_y actual, ya que al estar ordenado por x ascendente, cualquier punto previo ya tiene x menor o igual.

Algorithm 3 Alg3MiEstructura – Optimización mediante Ordenamiento

```
1: function ALG3MIESTRUCTURA(S)
2:     if S = ∅ then
3:         return ∅
4:     end if
5:     S' ← ordenar S por x ascendente (desempate por y ascendente)
6:     Pareto ← ∅
7:     min_y ← +∞
8:     for cada p = (x, y) ∈ S' do
9:         if y < min_y then
10:            Pareto ← Pareto ∪ {p}
11:            min_y ← y
12:        end if
13:    end for
14:    return Pareto
15: end function
```

Complejidad: $O(n \log n)$, dominada por el costo del ordenamiento inicial. El recorrido posterior es $O(n)$, por lo que la estructura de datos elegida (arreglo ordenado) permite evitar las comparaciones cuadráticas de dominancia de los algoritmos anteriores.

3. Análisis de Complejidad

La Tabla 1 resume el mejor caso, peor caso y complejidad teórica de cada algoritmo, considerando como operación relevante la comparación de dominancia $p \prec q$.

Cuadro 1: Comparación de complejidad teórica de los algoritmos

Algoritmo	**Mejor Caso**	**Peor Caso**	**Complejidad Teórica**
Alg1	$O(n^2)$	$O(n^2)$	$\theta(n^2)$
Alg2Arreglo	$O(n)$	$O(n^2)$	$O(n^2)$
Alg3MiEstructura	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

- **Peor caso:** ocurre cuando todos los puntos son parte de la frontera de Pareto (por ejemplo, puntos ubicados en una diagonal descendente). En este escenario, Alg2Arreglo debe comparar cada punto nuevo contra un conjunto C que crece linealmente, degenerando a $O(n^2)$.
- **Mejor caso:** ocurre cuando un punto domina rápidamente a casi todos los demás, permitiendo descartes tempranos en Alg2Arreglo (manteniendo $|C|$ pequeño) y por tanto un costo cercano a $O(n)$.
- Alg1 no presenta variación entre mejor y peor caso, ya que siempre recorre completamente ambos bucles anidados, sin mecanismos de descarte temprano efectivos sobre el total de comparaciones.
- Alg3MiEstructura mantiene un comportamiento estable en todos los casos, ya que su costo está dominado por el ordenamiento inicial ($O(n \log n)$) independientemente de la configuración de los puntos.

4. Implementación

Los tres algoritmos fueron implementados en Python. A continuación se muestra el código fuente completo, junto con el script de pruebas unitarias y el script de experimentos utilizado para medir los tiempos de ejecución.

4.1. skyline.py – Implementación de los algoritmos

Listing 1: Implementaci'on de Alg1, Alg2Arreglo y Alg3MiEstructura

```
def dominates(p, q):
    """Retorna True si el punto p domina al punto q"""
    # p dominates q if p_i <= q_i for all i and p_i < q_i for at least one i
    is_better_or_equal = True
    is_strictly_better = False

    for pi, qi in zip(p, q):
        if pi > qi:
            is_better_or_equal = False
            break
        if pi < qi:
            is_strictly_better = True

    return is_better_or_equal and is_strictly_better


def alg1(points):
    """
    Algoritmo simple para encontrar la frontera Pareto (skyline)
    Complexity: O(n^2)
    """
    pareto_front = []
    n = len(points)

    for i in range(n):
        p = points[i]
        is_dominated = False
        for j in range(n):
            if i == j:
                continue
            q = points[j]
            if dominates(q, p):
                is_dominated = True
                break
        if not is_dominated:
            pareto_front.append(p)

    return pareto_front


def alg2_arreglo(points):
    """
```

```
    Algoritmo incremental usando array para encontrar los candiadatos de la front
    Worst case: O(n^2), pero mejor en practica.
    """
    C = []
    for p in points:
        to_remove = []
        is_p_dominated = False
        for q in C:
            if dominates(q, p):
                is_p_dominated = True
                break
            if dominates(p, q):
                to_remove.append(q)

        if not is_p_dominated:
            # Remove all q in C that p dominates
            for item in to_remove:
                C.remove(item)
            C.append(p)
    return C


def alg3_mi_estructura(points):
    """
    Algoritmo optimizado (Caso 2D)
    Ordena asendentemente por X, y luego por Y.
    Guardar el minimo en Y hasta el momento
    Complexity: O(n log n) Dado el ordenamiento.
    """
    if not points:
        return []

    # Ordena por X (asc), luego por Y (asc)

    sorted_points = sorted(points, key=lambda x: (x[0], x[1]))

    pareto_front = []
    min_y = float("inf")

    for p in sorted_points:
        # Dado que los puntos estan ordenados por X, solo necesitamos que Y sea e
        # que el mejor Y hasta el momento no sea dominado
        # Nota: si varios valores tienen el mismo valor de X, el con menor Y pued

        if p[1] < min_y:
            pareto_front.append(p)
            min_y = p[1]

    return pareto_front
```

4.2. test_skyline.py – Pruebas unitarias

Se verificó la correctitud de Alg1 utilizando un conjunto de puntos análogo al de la Figura 1 del enunciado, comparando el resultado obtenido contra el conjunto de puntos Pareto esperado.

Listing 2: Prueba unitaria de Alg1

```
from skyline import alg1

def test_alg1():
    # Puntos con estructura similar a la Figura 1 del enunciado
    points = [
        (0.5, 8),   # p1
        (1, 4),     # p2
        (2, 2),     # p3
        (4, 1),     # p4
        (3, 6),     # p5
        (4, 5),     # p6
        (6, 3),     # p7
        (7, 2),     # p8
        (2, 7),     # p9
        (7, 5)      # p10
    ]

    expected = [(0.5, 8), (1, 4), (2, 2), (4, 1)]
    result = alg1(points)

    print(f"Points: {points}")
    print(f"Expected: {expected}")
    print(f"Result: {result}")

    assert set(result) == set(expected), f"Expectados {expected}, per se obtuvier

    print("Test passed!")

if __name__ == "__main__":
    test_alg1()
```

4.3. experiments.py – Script de experimentos

Este script genera conjuntos de puntos aleatorios de distintos tamaños y mide el tiempo de ejecución de cada uno de los tres algoritmos.

Listing 3: Script de generaci'on de datos y medici'on de tiempos

```
import time
import random
from skyline import alg1, alg2_arreglo, alg3_mi_estructura

def generate_points(n, type='random'):
    if type == 'random':
        return [(random.random(), random.random()) for _ in range(n)]
    elif type == 'worst':
        # Varios puntos en la frontera pareto: Y = 1/X curva o diagonal descendie
```

```
        return [(i, n - i) for i in range(n)]
    return []

def run_experiment():
    sizes = [1000, 5000, 10000]  # Mantener bajo para evitar tiempos excesivos de

    results = {}

    for n in sizes:
        print(f"Testing n={n}...")
        points = generate_points(n, 'random')

        # Alg 1
        start = time.time()
        alg1(points)
        t1 = time.time() - start

        # Alg 2
        start = time.time()
        alg2_arreglo(points)
        t2 = time.time() - start

        # Alg 3
        start = time.time()
        alg3_mi_estructura(points)
        t3 = time.time() - start

        results[n] = (t1, t2, t3)
        print(f"  n={n}: Alg1={t1:.4f}s, Alg2={t2:.4f}s, Alg3={t3:.4f}s")

    return results

if __name__ == "__main__":
    run_experiment()
```

5. Resultados Experimentales

Se realizaron pruebas con puntos aleatorios en el rango $[0, 1]$, midiendo el tiempo de ejecución (en segundos) de cada algoritmo para distintos tamaños de entrada n. Los resultados se muestran en la Tabla 2.

Cuadro 2: Tiempos de ejecución (segundos) para puntos aleatorios uniformes

n	**Alg1 (s)**	**Alg2Arreglo (s)**	**Alg3MiEstructura (s)**
1.000	0.0142	0.0022	0.0005
5.000	0.0535	0.0048	0.0034
10.000	0.1220	0.0064	0.0082

Se observa que, para conjuntos de puntos distribuidos uniformemente al azar, tanto Alg2Arreglo como Alg3MiEstructura superan ampliamente a Alg1 a medida que n crece, consistente con el análisis teórico presentado en la Sección 3.

6. Conclusiones

- El algoritmo ingenuo (Alg1) es ineficiente para grandes volúmenes de datos, ya que su comportamiento cuadrático no depende de la distribución de los puntos.
- El algoritmo incremental con arreglo (Alg2Arreglo) es muy superior en la práctica para datos distribuidos uniformemente, ya que el tamaño del conjunto de candidatos C se mantiene pequeño en comparación con n.
- La optimización mediante ordenamiento (Alg3MiEstructura) garantiza un rendimiento predecible y óptimo de $O(n \log n)$ para el caso bidimensional, independiente de la configuración de los puntos de entrada.
- La elección del algoritmo más adecuado depende del contexto: si se requiere un rendimiento garantizado sin importar la distribución de los datos, Alg3MiEstructura es la opción más robusta; si se trabaja en dimensión mayor a 2 (donde el ordenamiento simple no aplica directamente), Alg2Arreglo sigue siendo una alternativa práctica y eficiente.

  El código fuente completo de este trabajo junto con instrucciones de ejecucion (readme.md) está disponible en el siguiente repositorio de GitHub: https://github.com/MatiasEng/TareasADA.